



TECHNICAL UNIVERSITY OF MUNICH

**SCHOOL OF COMPUTATION, INFORMATION AND
TECHNOLOGY
INFORMATICS**

Master's Thesis in Informatics

**DRAFT - Thesis Proposal:
Something With Usability in PIX:E**

Rai Trouvain



TECHNICAL UNIVERSITY OF MUNICH

**SCHOOL OF COMPUTATION, INFORMATION AND
TECHNOLOGY
INFORMATICS**

Master's Thesis in Informatics

DRAFT - Thesis Proposal:
Something With Usability in PIX:E
Thesis Proposal: Irgendetwas mit Usability
in PIX:E

Author:	Rai Trouvain
Supervisor:	Prof. Dr. Georg Groh
Advisors:	Julian Geheeb M.Sc.
Submission Date:	DRAFT - 18.02.2026

Disclaimer

I confirm that this Master's thesis is my own work and I have documented all sources and material used.

(Date)

(Rai Trouvain)

Acknowledgements

The acknowledgements

Abstract

English Abstract

Zusammenfassung

German Abstract

Contents

1 Introduction	2
2 Background	3
3 Basic Path-Based Diagrams	4
3.1 Path Selection	4
3.1.1 Functionality	4
3.1.2 Implementation	4
3.1.3 Workflow	4
3.2 Diagram Generation	5
3.2.1 Functionality	5
3.2.2 Implementation	6
3.2.3 Workflow	6
3.2.4 Limitations	6
Bibliography	7

1 Introduction

2 Background

3 Basic Path-Based Diagrams

- basic idea already present in PaceMaker: select path in statechart and visualize data along path
- main design guideline: build upon high configurability already present in the system
- following sections describe functionalities, implementation choices and workflow/usage of these features implemented in pixe

3.1 Path Selection

3.1.1 Functionality

- selection of nodes in statechart triggers calculation of path connecting them
- selected path is highlighted (visual feedback)
- path is subsequently used for diagram generation

3.1.2 Implementation

- select start + end node, optionally intermediate -> calculate path using dijkstra
- `pathsApi.ts`: to allow for different algorithms in the future (e.g. specific pathfinding for nestedness and concurrency)

3.1.3 Workflow

- selection of at least two nodes
 - Ctrl + click
- path is calculated along nodes **in order of selection**
- if path is found, the corresponding nodes are highlighted
- path can be de-selected by removing node selection (clicking anywhere)

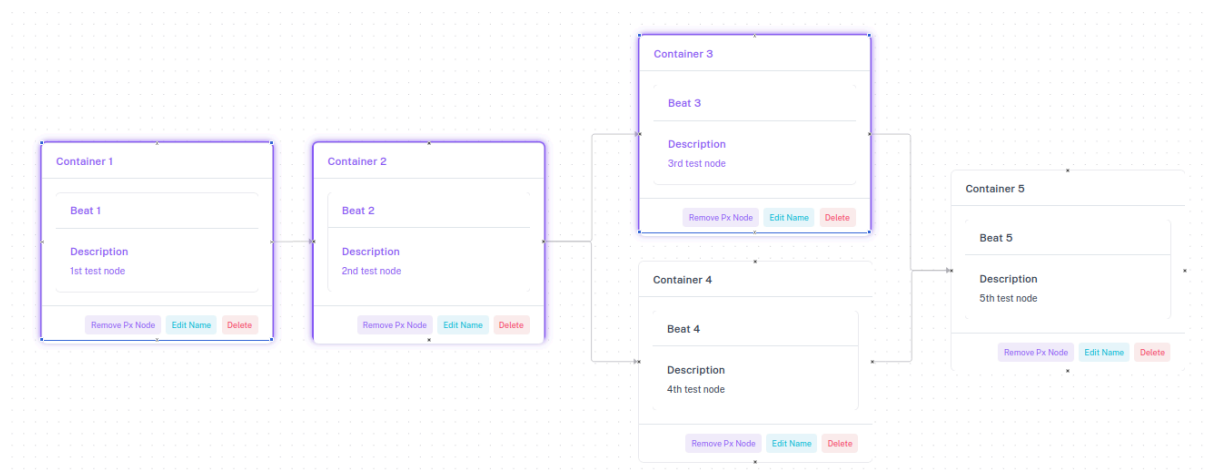


Figure 1: Path highlighted in state chart

3.2 Diagram Generation

3.2.1 Functionality

- expandable element on statechart page
 - usability: users can hide diagrams while working on statechart
- can create multiple diagrams and remove diagrams (red because diagrams do not persist)

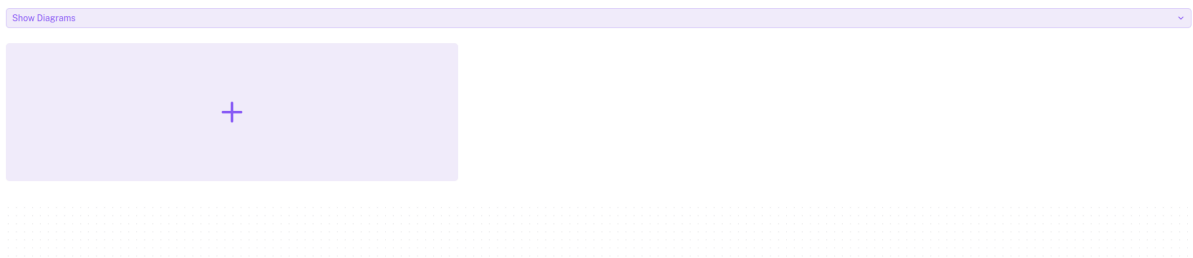


Figure 2: Upper section of the charts page with expandable element and UI for diagram creation

- selection of one or multiple component definitions for the y axis
- optionally selection of one component for the x axis (e.g. estimated playtime)
 - values are summed up along selected path
 - default: equal spacing

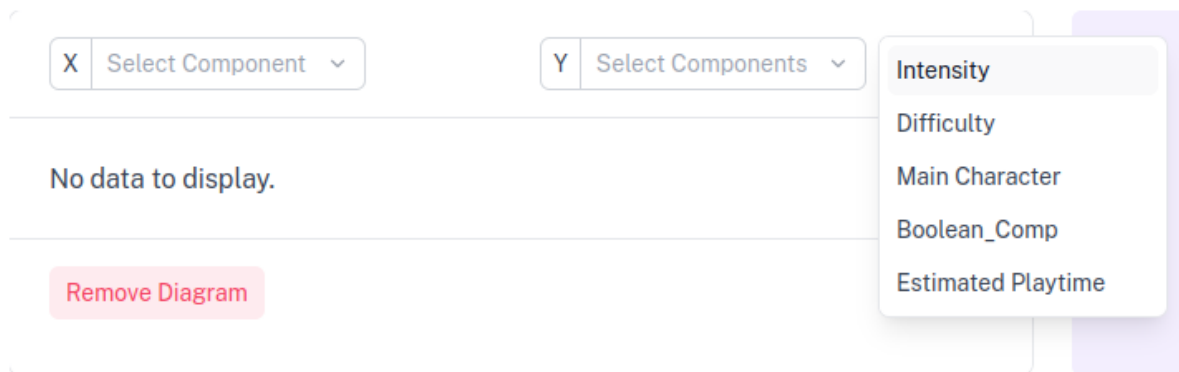


Figure 3: Selectors for components to be mapped to X/Y axes

- selected path along x axis
 - if no path selected: all nodes,  sorted



Figure 4: Diagram visualizing component along selected path

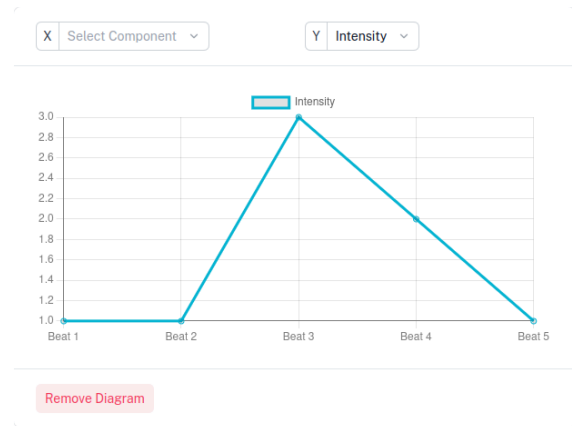


Figure 5: Diagram visualizing component across all nodes

3.2.2 Implementation

- [chartJS](#) library
- modular architecture: `PxDiagrams` as wrapper element
 - manages diagrams
 - currently: passes path (reactively), deletes diagrams
 - future work: path snapshots, different data types
- data is extracted from nodes once per diagram, switching between axis configurations is then done via different parsing configurations

3.2.3 Workflow

1. add new diagram
2. in any order:
 - select one or no component for x axis
 - select one or more component for y axis
 - select path
- changes in any of the configurations will be reflected live in the diagram

3.2.4 Limitations

- diagrams do not persist (reset when reloading page)
- only line diagrams for now
- same path selection for all diagrams

Bibliography